PROJECT README

# Jarvis - Personal AI Agent

An AI assistant that plans, uses tools safely and hands back finished work.

| | |
|---|---|
| **Type** | Personal project |
| **Year** | 2026 - ongoing |
| **Role** | Designer and developer |
| **Repository** | github.com/Masum-jubayel/jarvis-ai-agent |
| **Live demo** | To be added |

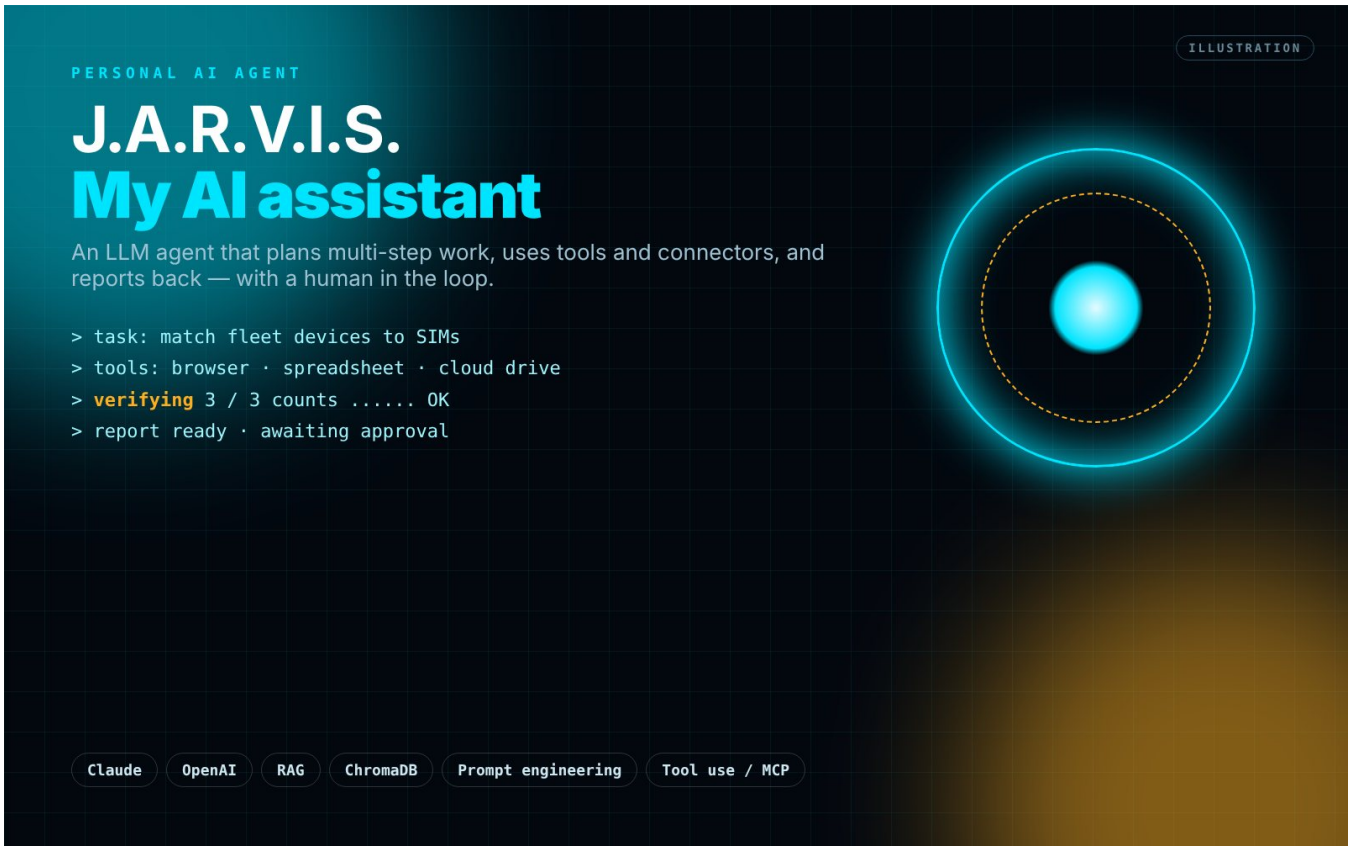



Illustration of the Jarvis interface.

## Contents

1. What the project is 2. Why it was built 3. How it works 4. Features 5. Architecture 6. Technologies 7. Security & privacy 8. Results 9. Next steps

## 1. What the project is

Jarvis is a personal AI agent built around large language models. It understands plain-language requests, remembers context, searches my own documents and uses tools - a browser, files, spreadsheets and scripts - to complete multi-step work such as preparing reports, auditing records or drafting documents.

## 2. Why it was built

Much of my day is repeatable work spread across many web portals and files. Chat assistants answer questions but do not act. I wanted an assistant that can plan, act safely and return a finished result for me to approve.

## 3. How it works, step by step

| Step | Stage | What happens |
|---|---|---|
| 1 | **Understand** | The request is analysed and broken into steps by the language model. |
| 2 | **Recall** | Relevant notes and procedures are retrieved from ChromaDB by semantic search (RAG). |
| 3 | **Act** | The model calls a tool: open a page, read a table, process a spreadsheet or run a script. |
| 4 | **Check** | Each result is compared with what was expected; errors trigger a retry or a question. |
| 5 | **Confirm** | Anything irreversible - sending, submitting, deleting - waits for my approval. |
| 6 | **Deliver** | The finished file or answer is returned, with a log of every action. |

## 4. Features

- Agent loop with tool calling
- Retrieval-augmented generation over personal documents
- Reusable skills: written procedures for recurring jobs
- Browser, file, spreadsheet and script tools
- Confirmation gates and action logs
- HUD-style interface in cyan and gold

## 5. Architecture

A planner (LLM) sits in the middle. On one side is memory (ChromaDB embeddings of documents and skills); on the other are tools (browser automation, file system, Python data tools). A guard layer checks every proposed action before it runs.

## 6. Technologies used

Each technology and the job it does in this project:

| Technology | What it is used for |
|---|---|
| **Python** | General-purpose programming language used for data processing, automation scripts and back-end logic. |
| **Large language models (Claude, OpenAI)** | AI models that understand and write natural language; used here to plan tasks and decide which tool to use next. |
| **Prompt engineering** | Writing clear instructions, examples and constraints so the model behaves reliably. |
| **Tool / function calling** | Lets the model request an action (open a page, read a file, run a script) and receive the result. |
| **RAG (retrieval-augmented generation)** | Before answering, relevant passages are retrieved from your own documents and given to the model, so answers are grounded in real information. |
| **ChromaDB** | Open-source vector database that stores text embeddings and finds the most similar passages quickly. |
| **Embeddings & semantic search** | Text is converted to numeric vectors so passages can be found by meaning, not just keywords. |
| **NLP** | Natural-language processing: techniques for understanding and generating human language. |
| **Pandas / NumPy** | Python libraries for working with tables and numbers efficiently. |
| **Browser automation** | Scripts that operate web portals the way a person would (search, read tables, capture exports) when no API is available. |
| **Model Context Protocol (MCP)** | An open standard for connecting AI assistants to tools and data sources in a consistent, permissioned way. |

## 7. Security & privacy

- Least privilege: each tool reaches only what it needs.
- No passwords or keys in prompts or memory.
- Human confirmation before irreversible actions.
- Complete action logs for review.

## 8. Results

- Recurring multi-portal jobs start from one sentence.
- Answers are grounded in my own documents.

## 9. Next steps

- Voice interface
- Scheduled jobs with daily briefings

Author: Masum Jubayel - Network, Security & IT Engineer. Portfolio: masum-jubayel.vercel.app | GitHub: github.com/Masum-jubayel